Decentralized Learning in Online Queuing Systems

Flore Sentenac

ENSAE Paris

Etienne Boursier

ENS Paris-Saclay

Vianney Perchet

ENSAE Paris
Criteo AI Lab

June 28th, 2021
Game Theory Seminar

Outline

- Introduction and model
- Existing results
- The case for cooperation
- Decentralized learning strategy

Introduction and model

Motivations

- traffic routing (road network, servers network)
- repeated congestion games
- cars (packets) taking same road (server) interfere

Learning in repeated games

Learning game environment (e.g. server/road quality)
Strategic interest (minimize own cost)

(Hart and Mas-Colell, 2000)

In repeated games, no internal regret strategies converge (in average) to the set of correlated equilibria.

$\rightarrow$ simple policies with strategic and learning guarantees

Learning in repeated games

Learning game environment (e.g. server/road quality)
Strategic interest (minimize own cost)

(Hart and Mas-Colell, 2000)

In repeated games, no internal regret strategies converge (in average) to the set of correlated equilibria.

$\rightarrow$ simple policies with strategic and learning guarantees

Assumption: outcome of each round is independent of the past (except in learning of agents)

Carryover feature

Road traffic
independence of rounds

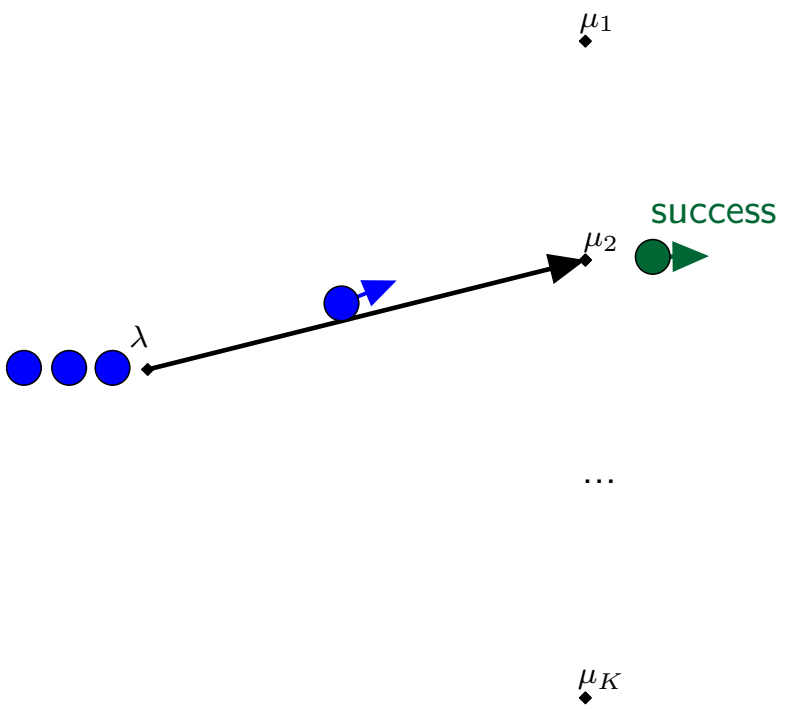
Second-by-second packet routing
Dropped packets have to be resent in next rounds

→ Learning in repeated games **with carryover**?

Model: single queue

At each $t = 1, \ldots, \infty$

- packet arrives with proba λ
- sends a packet to server $k \in [K]$
- server k clears with proba μ_k
- if fails $\rightarrow$ packet back in queue

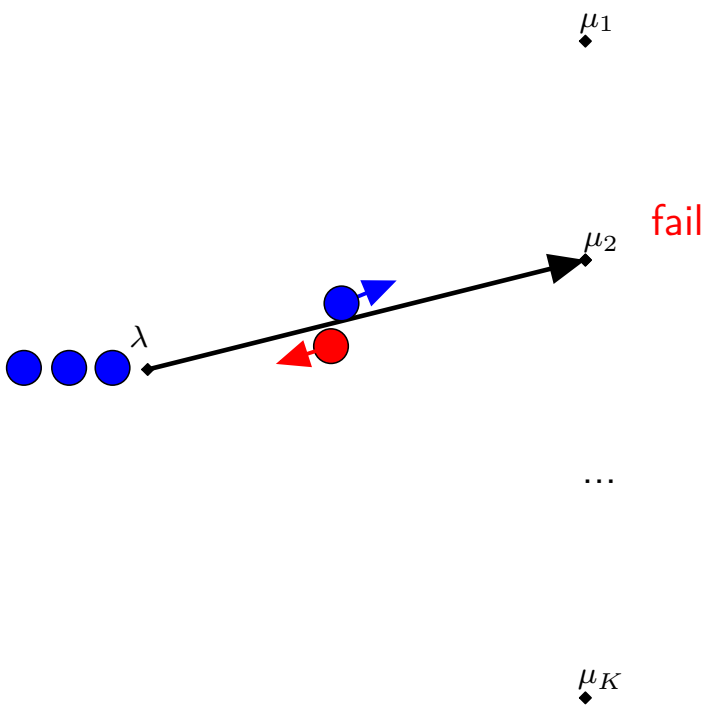


$\rightarrow$ **multi-armed bandits approach**

Model: single queue

At each $t = 1, \ldots, \infty$

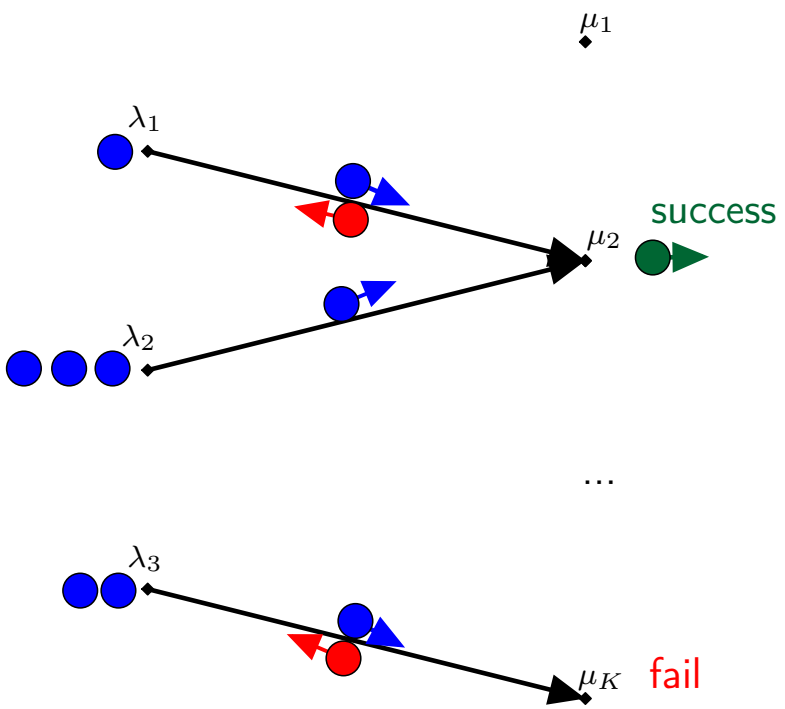
- packet arrives with proba λ
- sends a packet to server $k \in [K]$
- server k clears with proba μ_k
- if fails $\rightarrow$ packet back in queue



$\rightarrow$ **multi-armed bandits approach**

Model: multiple queues

- N queues ($N \leq K$)
- Heterogeneous arrival rates λ_i
- each queue chooses $a_t^i \in [K]$
- Server treats one packet at a time



related to multiplayer/competing bandits

Stability

Q_t^i number of packets in queue i at time t

A queue i is **stable** if for any r, there is a constant $C_r > 0$ such that

$$\mathbb{E}[(Q_t^i)^r] \leq C_r \qquad \forall t \in \mathbb{N}$$

System is stable if all queues are stable

Notation: $\lambda_{(1)} \geq \lambda_{(2)} \geq \ldots \geq \lambda_{(N)}$ and $\mu_{(1)} \geq \ldots \geq \mu_{(K)}$
Define slack

$$\eta = \max \left\{ \eta' \in \mathbb{R}_+ \mid \forall n \in [N], \eta' \sum_{i=1}^{n} \lambda_{(i)} \leq \sum_{i=1}^{n} \mu_{(i)} \right\}$$

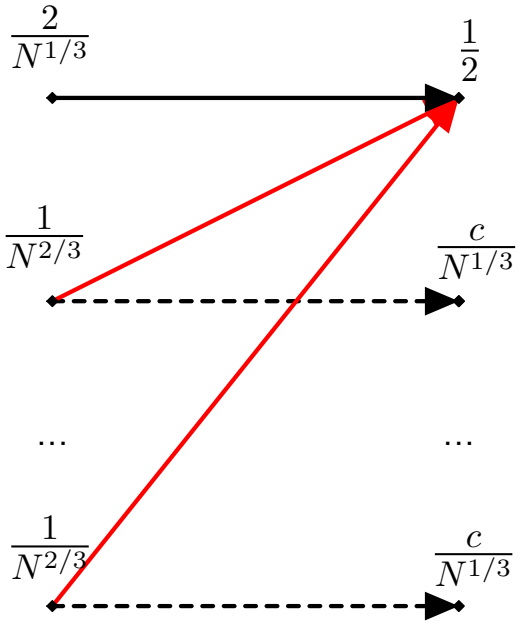
Centralized case: there is a stable strategy iff $\eta > 1$.

Goal: decentralized stable strategies for small η

Priority choice

A server can treat only one packet at a time.
Which packet to choose?

At random?
$\rightarrow$ unstable Nash equilibria with large η ($\gtrsim N^{1/3}$)



$N = K$
$\lambda_1 = \frac{2}{N^{1/3}}$ and $\lambda_i = \frac{1}{N^{2/3}}$ for all $i \geq 2$
$\mu_1 = \frac{1}{2}$ and $\lambda_i = \frac{c}{N^{1/3}}$ for all $i \geq 2$

queue 1 cannot clear

Priority choice

A server can treat only one packet at a time.
Which packet to choose?

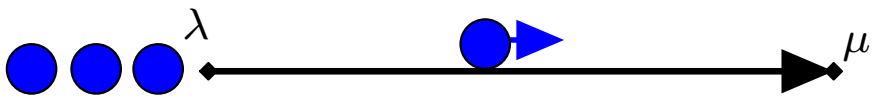
Treat oldest packet
$\rightarrow$ force better Nash equilibria
$\rightarrow$ carryover effect

if some queue accumulates packets $\rightarrow$ gets priority
bad performance for other queues on the long run $\rightarrow$ incites to cooperation

Existing results

Centralized case

Single queue, single server



Random walk (with frontier at 0)

- $\lambda < \mu$ $\rightarrow$ negative bias, stable
- $\lambda = \mu$ $\rightarrow$ no bias, queue size grows in $\sqrt{t}$
- $\lambda > \mu$ $\rightarrow$ positive bias, queue size in $(\lambda - \mu)t$

$\Longrightarrow$ centralized strategy stable iff $\eta > 1$

No regret strategies

Define regret

$$R_i(T) = \max_{k \in [K]} \sum_{t=1}^{T} \nu_t^i(k) - \sum_{t=1}^{T} \overbrace{\nu_t^i(a_t^i)}^{\text{1 if clears}}$$

Generally, learning agents selfishly minimize their regret

- no regret strategies known (eg EXP.3.P)
- used without strategic considerations
- converge in average to equilibria (without carryover effect)

No regret strategies

(Gaitonde and Tardos, 2020a)

If $\eta > 2$ and all queues follow a no regret strategy: $R_i(T) = o(T)$ w.h.p., then the system is stable.

$\rightarrow$ good when slack of the system is large

But minimizing regret is myopic and disregards long terms effect (carryover)
Can we do better?

Patient game

Define game $\mathcal{G} = ([N], (c_i)_{i=1}^n, \boldsymbol{\mu}, \boldsymbol{\lambda})$ with

Action Space: $p_i \in \mathcal{P}([K])$

Cost Function: All queues choose their server $a_t^i \sim p_i$ at each time step and

$$c_i(p_i, \boldsymbol{p_{-i}}) = \lim_{t \to +\infty} \frac{T_t^i}{t}$$

- T_t^i is the age of the oldest packet in queue i at time t
- this limit exists (deterministically)
- queue i is stable $\implies c_i(p_i, \boldsymbol{p_{-i}}) = 0$

Patient game

(Gaitonde and Tardos, 2020b)

If $\eta > \frac{e}{e-1}$, all Nash equilibria of $\mathcal{G}$ are stable.

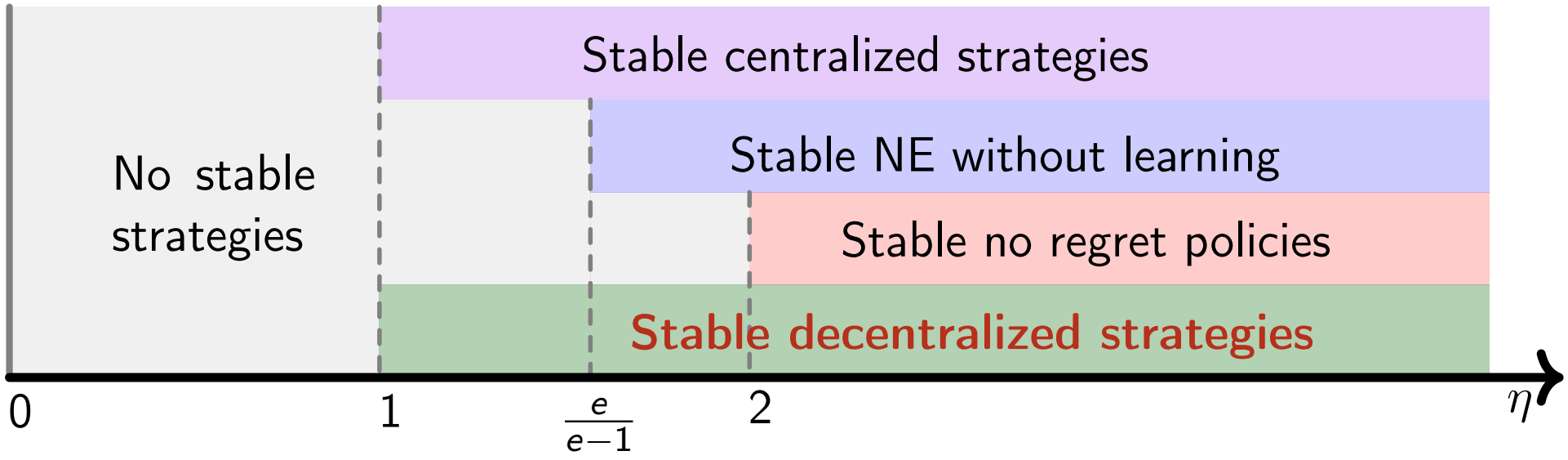
- long term considerations lead to stable strategies for $\eta \leq 2$
- $\frac{e}{e-1}$ is the *"price of anarchy"*
- reachable for learning agents?

Our contributions

1. cooperation required for stability of learning queues with $\eta \leq 2$
2. propose decentralized learning algorithm ADeQuA

Theorem

If $\eta > 1$ and all queues follow ADeQuA*, then the system is stable.*



The case for cooperation

A patient regret

Regret ignores the long term effects and leads to myopic decisions

$$\mathbb{E}[R_i^{\text{ext}}(T)] := \max_{p \in \mathcal{P}([K])} \sum_{t=1}^{T} \overbrace{\mathbb{E}_{\tilde{a}_t^i \sim p}[\nu_t^i(a_{1:t-1}^i, \tilde{a}_t^i)]}^{\text{reward we would get by playing } p \textbf{ for this round}} - \sum_{t=1}^{T} \overbrace{\mathbb{E}[\nu_t(a_{1:t}^i)]}^{\text{actual reward}}$$

A patient regret

Regret ignores the long term effects and leads to myopic decisions

$$\mathbb{E}[R_i^{\text{ext}}(T)] := \max_{p \in \mathcal{P}([K])} \sum_{t=1}^{T} \overbrace{\mathbb{E}_{\tilde{a}_t^i \sim p}[\nu_t^i(a_{1:t-1}^i, \tilde{a}_t^i)]}^{\text{reward we would get by playing } p \textbf{ for this round}} - \sum_{t=1}^{T} \overbrace{\mathbb{E}[\nu_t(a_{1:t}^i)]}^{\text{actual reward}}$$

Policy regret instead takes into account the effects of an action on the environment

$$R_i^{\text{pol}}(T) := \max_{p \in \mathcal{P}([K])} \sum_{t=1}^{T} \overbrace{\mathbb{E}_{\tilde{a}_{1:t}^i \sim \otimes_{i=1}^t p}[\nu_t(\tilde{a}_{1:t}^i)]}^{\text{reward we would get if we were playing } p \textbf{ for all rounds}} - \sum_{t=1}^{T} \mathbb{E}[\nu_t(a_{1:t}^i)]$$

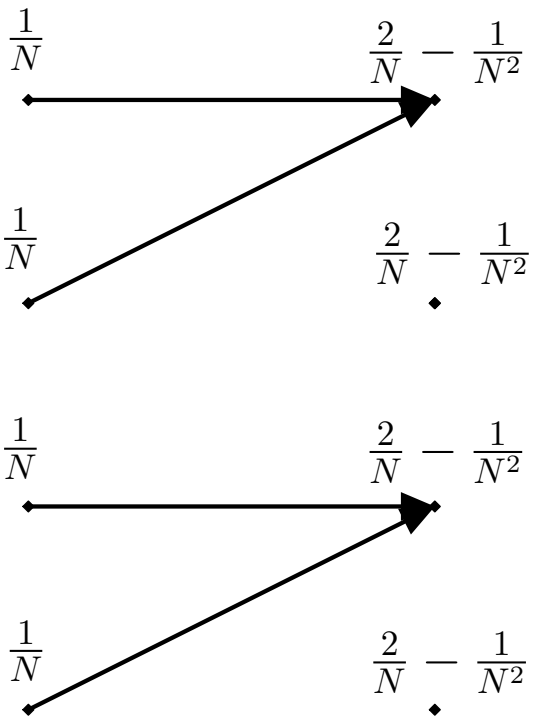
No policy regret strategies

Patient agents should minimize policy regret
Are no policy regret strategies more stable? **No**

There is an instance with $\eta = 2 - \mathcal{O}\left(\frac{1}{N}\right)$, s.t. each queue's policy regret is $o(T)$, but the system is not strongly stable.

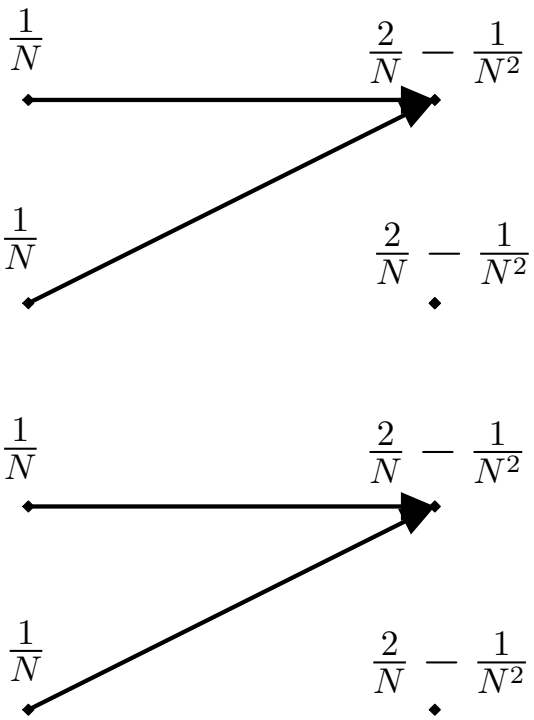
$\rightarrow$ no policy regret is *no better* than no regret

Counter Example



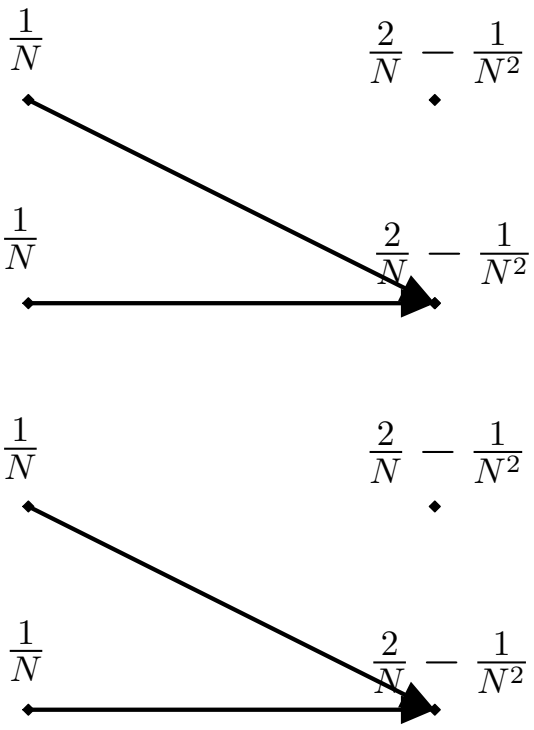
$\lambda_i = \frac{1}{N}$ for all i
$\mu_i = \frac{2}{N} - \frac{1}{N^2}$ for all i

Counter Example



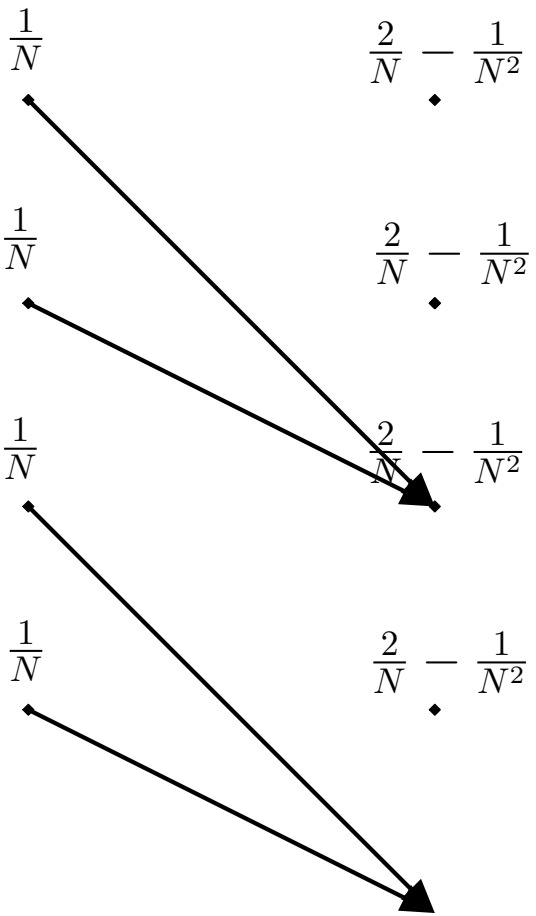
First phase of length αT
Pairwise actions

Counter Example (first phase)



First phase of length αT
Pairwise actions

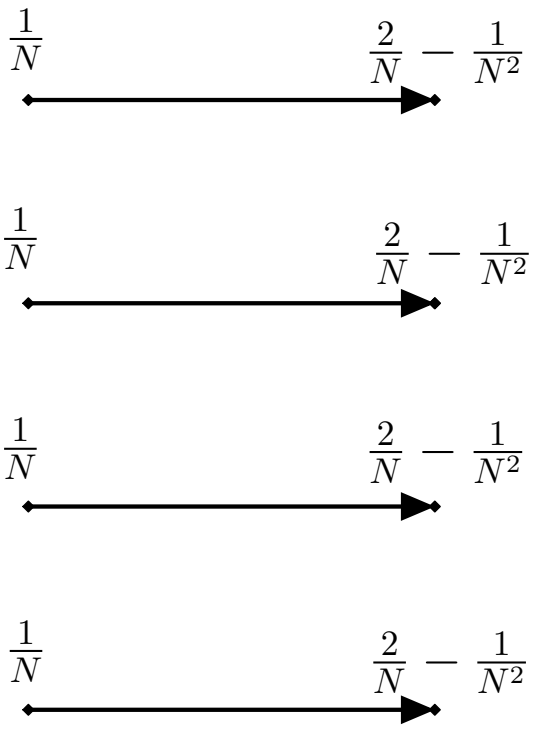
Counter Example (first phase)



First phase of length αT
Pairwise actions

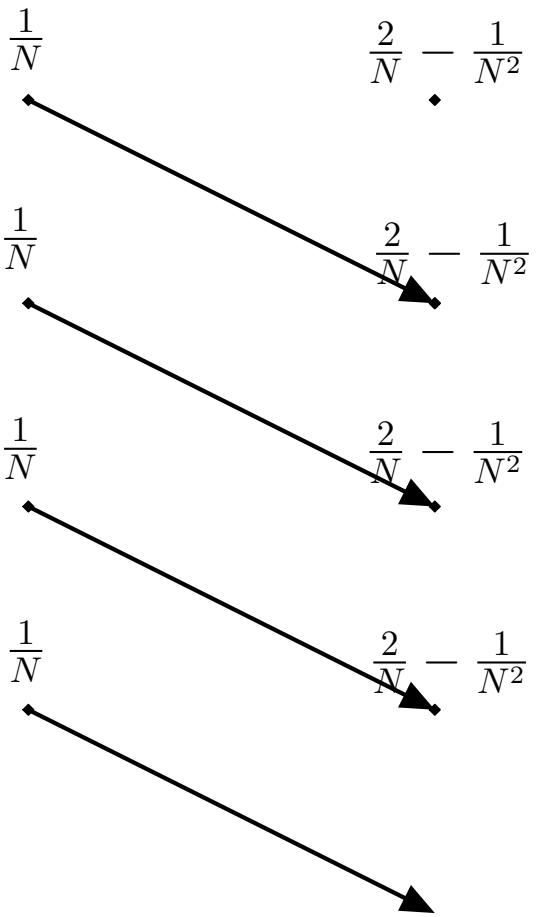
$\rightarrow$ accumulate packets during this phase

Counter Example (second phase)



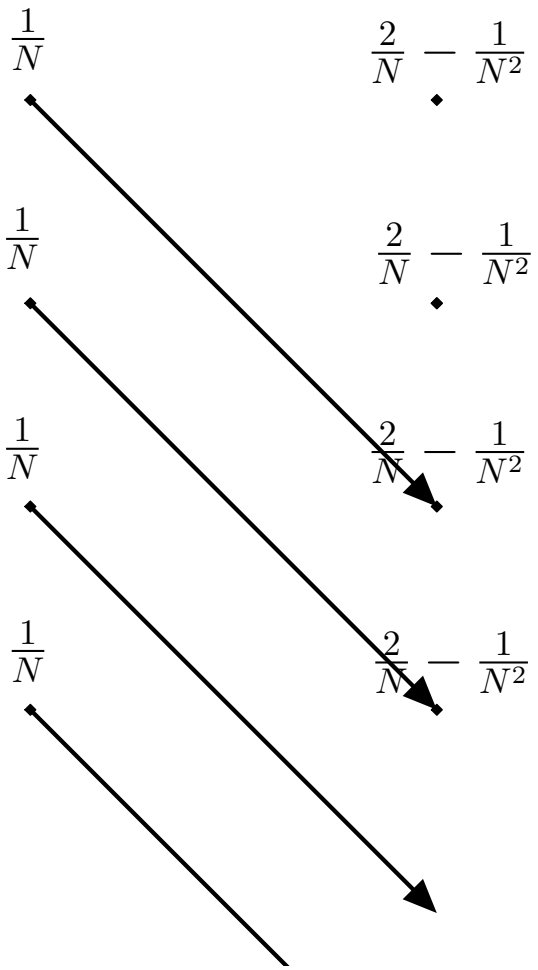
Second phase of length $(1-\alpha)T$
No collision

Counter Example (second phase)



Second phase of length $(1-\alpha)T$
No collision

Counter Example (second phase)

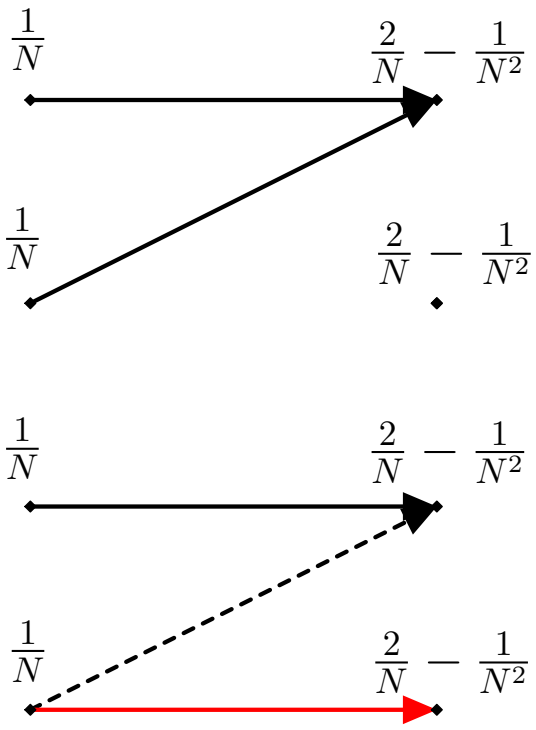


Second phase of length $(1-\alpha)T$
No collision

$\rightarrow$ clear packets during this phase
if α large enough, still accumulate overall $\Omega(T)$ packets
$\rightarrow$ unstable

Counter example (no policy regret)

What if queue i deviates and plays $p \in \mathcal{P}([K])$ at each round?

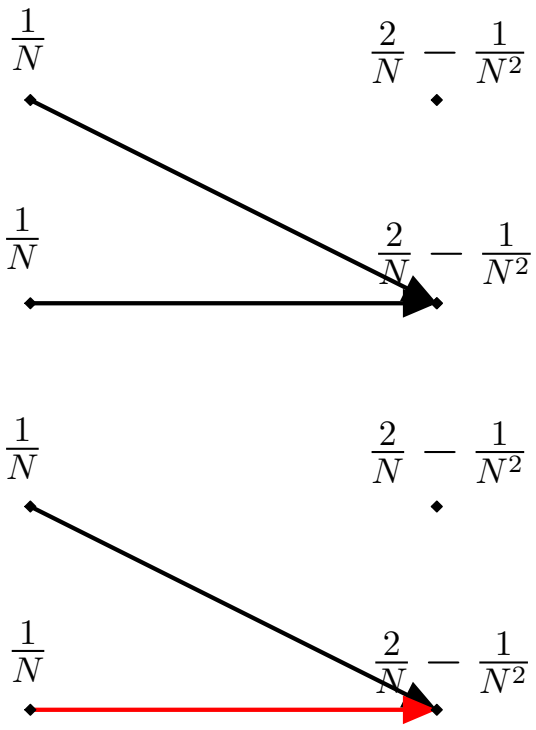


First phase

$\rightarrow$ clear all packets

Counter example (no policy regret)

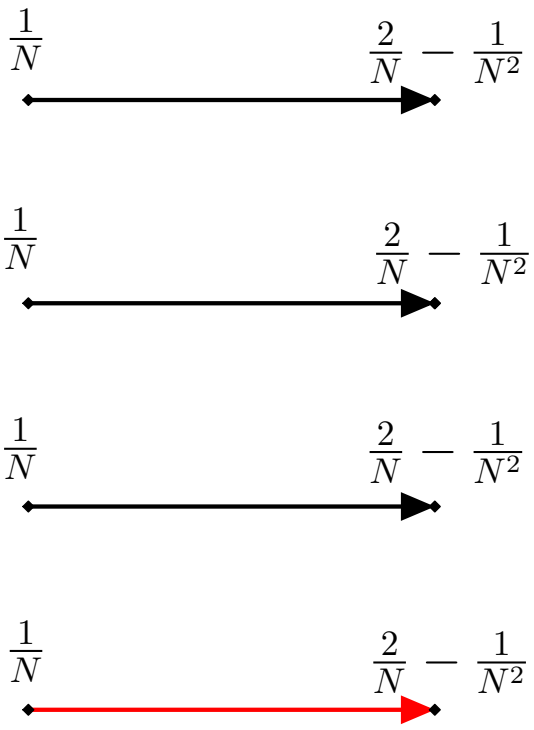
What if queue i deviates and plays $p \in \mathcal{P}([K])$ at each round?



First phase

$\rightarrow$ clear all packets

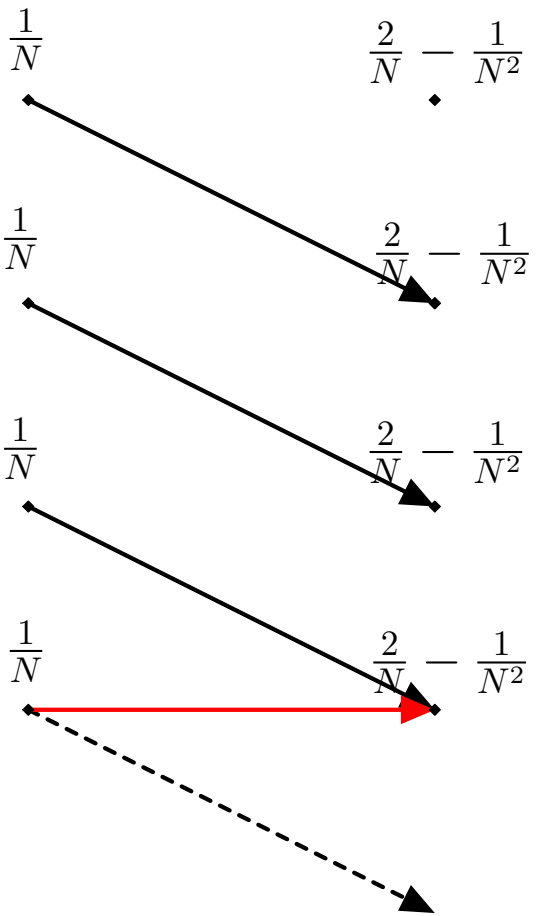
Counter example (no policy regret)



Second phase
many collisions
other queues have priority
accumulate $\Omega(T)$ packets

for α small enough, accumulate more packets when deviating
$\rightarrow$ No policy regret strategies!

Counter example (no policy regret)



Second phase
many collisions
other queues have priority
accumulate $\Omega(T)$ packets

for α small enough, accumulate more packets when deviating
$\rightarrow$ No policy regret strategies!

Case for cooperative algorithm

for some α, strategy profile is both unstable and no policy regret
$\rightarrow$ patience not enough for learning agents with $\eta < 2$
Counter example shows cooperation/coordination is also required

Case for cooperative algorithm

for some α, strategy profile is both unstable and no policy regret
$\rightarrow$ patience not enough for learning agents with $\eta < 2$
Counter example shows cooperation/coordination is also required

Centralized strategy when $\eta > 1$: play same joint distribution at each round
$\rightarrow$ correlated equilibria of $\mathcal{G}$
$\rightarrow$ *"correlated price of stability"* $= 1$

Design cooperative learning strategies!

Decentralized learning strategy

A stable learning strategy

Assumptions:

- all queues know N and have attributed ranks $i \in [N]$
- shared randomness between queues

Theorem

If $\eta > 1$ and all queues follow ADeQuA, *then the system is stable.*

Inspired by ε-greedy policy

```
for t = 1, ..., ∞ do
    Draw ω_1 ~ Bernoulli(ε_t)          // shared randomness
    if ω_1 = 1 then Explore
    else Exploit
end
```

ADeQuA: A DEcentralized QUeuing Algorithm

Exploration

All queues explore simultaneously and explore either $\boldsymbol{\mu}$ or $\boldsymbol{\lambda}$

Explore $\boldsymbol{\mu}$: queues choose servers without colliding
$\rightarrow$ accurate estimations of all μ_k

Assumption: servers break ties in packets' age uniformly at random

Explore $\boldsymbol{\lambda}$: when queue i explores queue j, both choose same server k **with packet generated at t (if it exists)**
i clears with probability $(1 - \frac{\lambda_j}{2})\mu_k \rightarrow$ estimate λ_j

Exploitation: notations

$\mathfrak{B}_K =$ set of bistochastic matrices
$\mathfrak{S}_K =$ set of permutations (or permutation matrices) on $[K]$

Dominant mapping $\phi : \mathbb{R}^N \times \mathbb{R}^K \to \mathfrak{B}_K$ such that for any (λ, μ):

$$\lambda < P\mu \qquad \text{if possible}$$

! **Birkhoff von Neumann decomposition** $\psi : \mathfrak{B}_K \to \mathcal{P}(\mathfrak{S}_K)$ such that for any P, $\mathbb{E}[\psi(P)] = P$

For convenience, $\psi(P) : [0, 1] \to \mathfrak{S}_K$

Exploitation: centralized

When centralized:

- $\phi : (\hat{\lambda}, \hat{\mu}) \mapsto$ marginals ensuring stability
- $\psi : P \mapsto$ coupling without collision

Draw $\omega_2 \sim \mathcal{U}(0,1)$	`// shared randomness`
Play $\psi(\phi(\hat{\lambda}, \hat{\mu}))(\omega_2)$	

Centralized exploitation

When decentralized:

- compute mapping $\hat{A}^i = \psi(\phi(\hat{\lambda}^i, \hat{\mu}^i))$
- play $\hat{A}^i(\omega_2)(i)$

Exploitation: need for regularity

$\hat{A}^i$ computed from own estimates $(\hat{\lambda}^i, \hat{\mu}^i)$, which differ (but are close)
General dominant mappings and BvN decompositions are non-continuous

$\|\hat{A}^i - \hat{A}^j\|$ arbitrarily large $\implies$ too many collisions

If ϕ and ψ **regular** $\rightarrow \|\hat{A}^i - \hat{A}^j\|$ small
$\implies$ small amount of collisions

Goal: Design **regular** dominant mapping and BvN decomposition

Dominant mapping

Goal $\phi : \mathbb{R}^N \times \mathbb{R}^K \to \mathfrak{B}_K$ such that for any (λ, μ):

$$\lambda < P\mu \qquad \text{if possible}$$

Usual dominant mappings sort λ and μ $\to$ discontinuity

$$\phi(\lambda, \mu) = \arg\min_{P \in \mathfrak{B}_K} \max_{i \in [N]} - \ln \left(\sum_{j=1}^{K} P_{i,j}\mu_j - \lambda_i \right) + \frac{1}{2K} \|P\|_2^2.$$

- locally Lipschitz objective
- strong convexity $\implies$ regularity of arg min
- optimization methods to approximate ϕ

Dominant mapping: regularity

Define margin $\Delta := \min_{k \in [N]} \frac{1}{k} \sum_{i=1}^{k} (\mu_{(i)} - \lambda_{(i)})$

$$\Delta > 0 \iff \eta > 1$$

Regularity of ϕ

If $\|(\hat{\lambda} - \lambda, \hat{\mu} - \mu)\|_\infty \lesssim \Delta$:

$$\|\phi(\hat{\lambda}, \hat{\mu}) - \phi(\lambda, \mu)\|_2 \lesssim \frac{K}{\Delta} \|(\hat{\lambda} - \lambda, \hat{\mu} - \mu)\|_\infty,$$

BvN decomposition

input : $\hat{P} \in \mathfrak{B}_K$
$j \leftarrow 1$
$G \leftarrow$ complete $K \times K$ bipartite graph
while $\hat{P} \neq \mathbf{0}$ **do**
  For all $(i,k) \notin \mathrm{supp}(\hat{P})$, remove edge (i,k) in G
  $P[j] \leftarrow$ perfect matching in G `// Hopcroft-Karp algorithm`
  $z[j] \leftarrow \min_{(i,k)\in \mathrm{supp}(P[j])} \hat{P}_{i,k}$ `// maximal weight we can remove`
  $\hat{P} \leftarrow \hat{P} - z[j]P[j]$ and $j \leftarrow j+1$
end
return $(z[j], P[j])_j$

Birkhoff algorithm

BvN decomposition of $\hat{P}$ (non-continuous)
computationally efficient: sequence of length $\leq K^2$

$$\psi(P)(\omega) = P[j_\omega]$$
where $\sum_{j \leq j_\omega} z[j] \leq \omega < \sum_{j \leq j_\omega + 1} z[j]$.

BvN decomposition

input : $\hat{P} \in \mathfrak{B}_K$
$j \leftarrow 1$
$G \leftarrow$ complete $K \times K$ bipartite graph
while $\hat{P} \neq \mathbf{0}$ **do**
  For all $(i,k) \notin \mathrm{supp}(\hat{P})$, remove edge (i,k) in G
  $P[j] \leftarrow$ perfect matching in G `// Hopcroft-Karp algorithm`
  $z[j] \leftarrow \min_{(i,k)\in \mathrm{supp}(P[j])} \hat{P}_{i,k}$ `// maximal weight we can remove`
  $\hat{P} \leftarrow \hat{P} - z[j]P[j]$ and $j \leftarrow j+1$
end
return $(z[j], P[j])_j$

Birkhoff algorithm

BvN decomposition of $\hat{P}$ (non-continuous)
computationally efficient: sequence of length $\leq K^2$

$$\psi(P)(\omega) = P[j_\omega]$$
where $\sum_{j \leq j_\omega} z[j] \leq \omega < \sum_{j \leq j_\omega + 1} z[j]$.

BvN decomposition

input : $\hat{P} \in \mathfrak{B}_K$, $C \in \mathbb{R}^{K \times K}$ (cost matrix)
$j \leftarrow 1$
while $\hat{P} \neq \mathbf{0}$ **do**
  For all $(i,k) \notin \mathrm{supp}(\hat{P})$, $C_{i,k} \leftarrow +\infty$ `// remove edge` (i,k)
  $P[j] \leftarrow \mathrm{Hungarian}(C)$ `// matching with minimal cost w.r.t.` C
  $z[j] \leftarrow \min_{(i,k) \in \mathrm{supp}(P[j])} \hat{P}_{i,k}$
  $\hat{P} \leftarrow \hat{P} - z[j]P[j]$ and $j \leftarrow j+1$
end
return $(z[j], P[j])_j$

Ordered Birkhoff algorithm

return $P[j]$ in increasing order

$$P[j] \prec_C P[j+1] \iff \sum_{i,k} C_{i,k} P[j]_{i,k} < \sum_{i,k} C_{i,k} P[j+1]_{i,k}$$

$$\psi(P)(\omega) = P[j_\omega]$$

where $\sum_{j \leq j_\omega} z[j] \leq \omega < \sum_{j \leq j_\omega + 1} z[j]$.

BvN decomposition: regularity

Proposition:
$z[j]$ = the j-th non-zero of sequence $(z_l(P))_{1 \leq l \leq K!}$ with

$$z_j(P) = \min_{(i,k) \in \mathrm{supp}(P_j)} \left(P - \sum_{l=1}^{j-1} z_l(P) P_l \right)_{i,k} \tag{1}$$

where (P_j) increasing sequence of permutation matrices

$\rightarrow$ continuity by induction

Regularity of ψ

$$\underbrace{\mathbb{P}_{\mathcal{U}(0,1)}(\psi(\hat{P}^i) \neq \psi(\hat{P}^j))}_{\geq \text{ probability of collision}} \leq 2^{2K^2} \|\hat{P}^i - \hat{P}^j\|_\infty.$$

Stability result

Take $\varepsilon_t = (N+K)t^{-\frac{1}{5}}$

Theorem (Stability)

If $\eta > 1$, for any queue i and any $r \in \mathbb{N}$, there exists a constant C_r depending only on r such that

$$\mathbb{E}[(Q_t^i)^r] \leq C_r K N \left(\frac{N^{\frac{5}{2}} K^{\frac{5}{2}} 2^{5K^2}}{(\min(1, K\bar{\mu}) \min_i \lambda_i)^{\frac{5}{4}} \Delta^5} \right)^r, \quad \textit{for all } t \in \mathbb{N}.$$

- stability with $\eta > 1$
- super exponential dependency due to BvN decomposition
- asymptotic number of packets similar to centralized case

Sketch of Proof

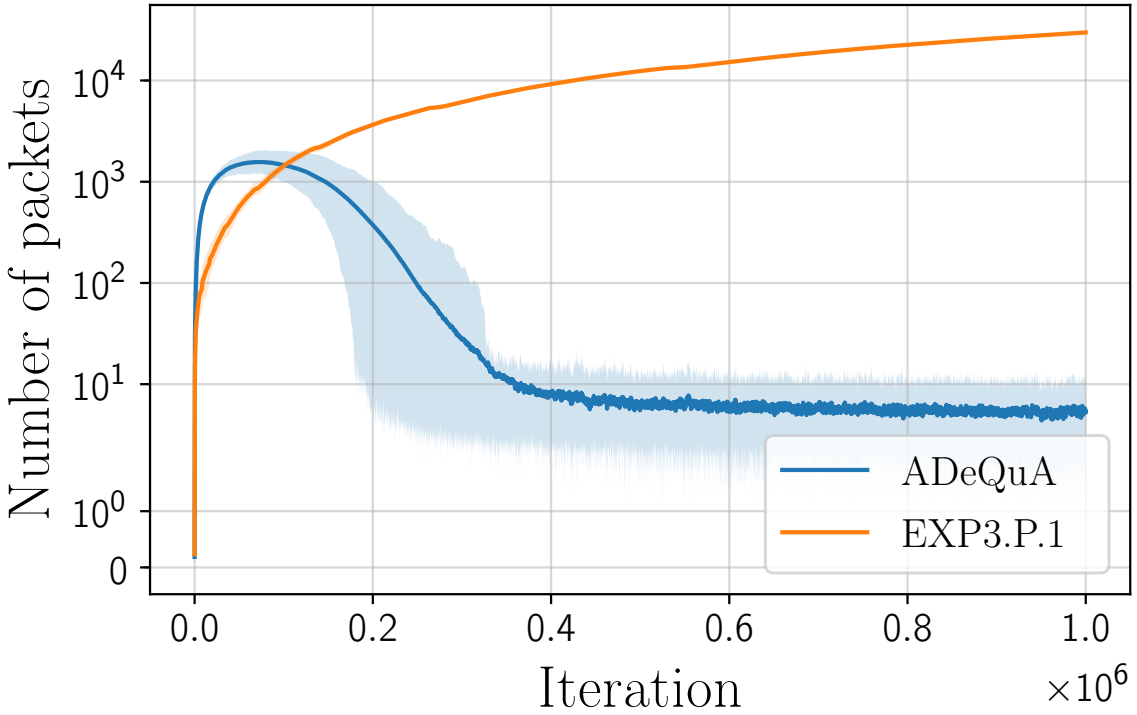
For $t \geq \overbrace{\tau}^{\text{learning time}}$, λ and μ are estimated tightly enough (concentration inequalities)

Afterwards, regularity of ϕ and ψ lead to

- small number of collisions
- queues clear with proba $\lambda_i + \mathcal{O}(\Delta)$

$\implies$ number of packets decreases after τ

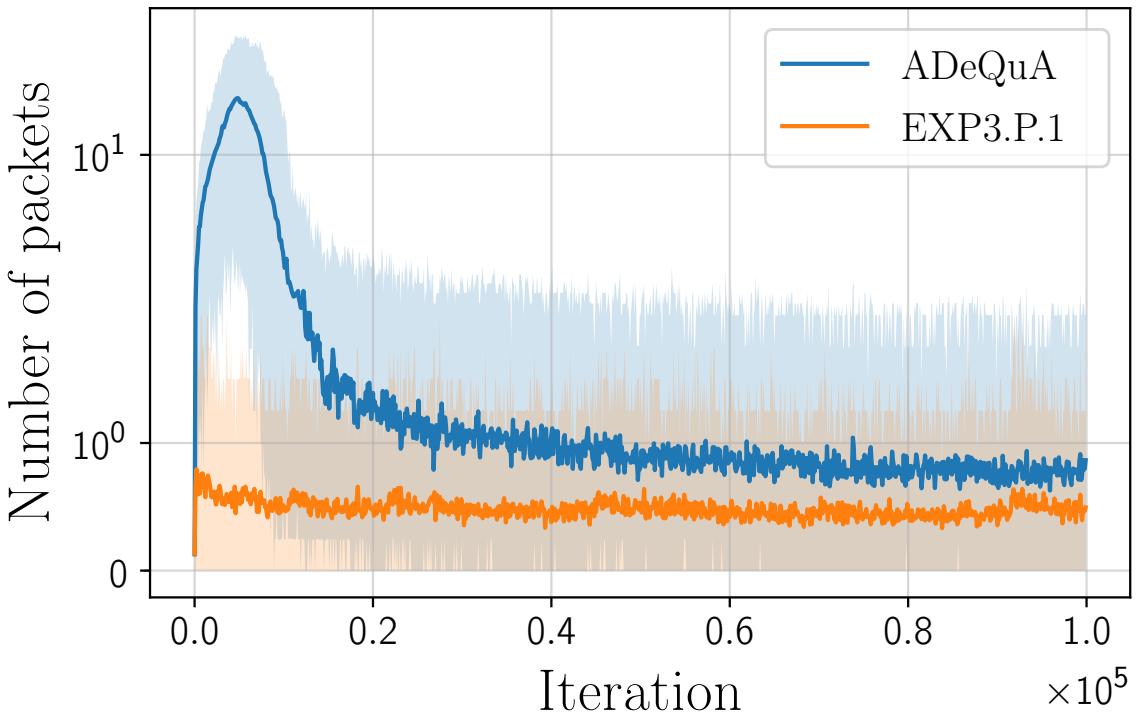
Simulations: hard instance



Hard instance, $\eta < 2$.

- No regret strategies: unstable
- ADeQuA: stable
  + number of packets decreases after learning

Simulations: easy instance



Easy instance, $\eta > 2$.

- both strategies stable
- No regret better suited to easy instances?

Recap

- carryover effect incites cooperation in queuing systems
- patience not enough for learning
- first decentralized stable learning strategy
- confirmed empirically

Open questions

- design of better dominant mapping and BvN decomposition
- dynamic case: queues don't have a common time clock
- enhancement with additional observation (eg collision information)

Thanks!